K.R. MANGALAM UNIVERSITY
THE COMPLETE WORLD OF EDUCATION

School of Engineering & Technology

K.R. Mangalam University

School of Engineering & Technology

Operating System Lab (ENCA252)

Lab Assignment 3

Comprehensive Implementation of Page Replacement Algorithms

Submitted by:

Name: Anurag Kumar Mishra

Roll Number: 2401201076

Course: BCA (AI & DS)

Submitted To:

Dr. Jyoti Yadav

Objective

This assignment implements and analyzes five classical page-replacement algorithms: **FIFO**, **LRU**, **Optimal**, **MRU**, and **Second Chance (Clock)**. Each algorithm is simulated on a common reference string to compare performance via page-fault counts, enabling a thorough understanding of virtual-memory management.

> ■ **Reference Input**
> Reference String: 7 3 0 3 2 1 2 0 7 0 1 2 | Number of Frames: 3

FIFO — First-In, First-Out

How It Works

FIFO evicts the page that has been in memory the longest. The page frames behave like a simple queue — the oldest arrival is always the first to be replaced when a new page needs to be loaded and no free frame is available.

Implementation Logic

A FIFO queue tracks arrival order. On each page fault (when all frames are occupied), the page at the head of the queue is evicted and the new page is appended at the tail. No usage frequency or recency is considered — purely order of arrival.

> ■ **Note**
> FIFO may suffer Belady's Anomaly: increasing the number of frames can sometimes increase the page-fault count rather than reduce it.

Step-by-Step Simulation Trace

Step	Page	Frame 1	Frame 2	Frame 3	Status
1	7	7	-	-	✗ Miss
2	3	7	3	-	✗ Miss
3	0	7	3	0	✗ Miss
4	3	7	3	0	✓ Hit
5	2	3	0	2	✗ Miss
6	1	0	2	1	✗ Miss
7	2	0	2	1	✓ Hit
8	0	0	2	1	✓ Hit
9	7	2	1	7	✗ Miss
10	0	1	7	0	✗ Miss
11	1	1	7	0	✓ Hit
12	2	7	0	2	✗ Miss

■ **Total Page Faults: 8**

LRU — Least Recently Used

How It Works

LRU evicts the page that was accessed least recently. The principle exploits temporal locality: pages used recently are statistically more likely to be needed again in the near future.

Implementation Logic

A timestamp or use-counter is maintained per loaded page. On a page fault the page with the oldest last-access time is evicted. While accurate, full hardware LRU is expensive; real systems use approximations like the Clock algorithm.

> ■ **Tip**
> LRU consistently approaches Optimal performance across diverse workloads and remains the most widely studied practical replacement policy.

Step-by-Step Simulation Trace

Step	Page	Frame 1	Frame 2	Frame 3	Status
1	7	7	-	-	✗ Miss
2	3	7	3	-	✗ Miss
3	0	7	3	0	✗ Miss
4	3	7	0	3	✓ Hit
5	2	0	3	2	✗ Miss
6	1	3	2	1	✗ Miss
7	2	3	1	2	✓ Hit
8	0	1	2	0	✗ Miss
9	7	2	0	7	✗ Miss
10	0	2	7	0	✓ Hit
11	1	7	0	1	✗ Miss
12	2	0	1	2	✗ Miss

■ Total Page Faults: 9

Optimal — Belady's Algorithm

How It Works

The Optimal algorithm replaces the page that will not be used for the longest period in the future (or never again). It achieves the theoretical minimum possible page-fault count for any given reference string and frame count.

Implementation Logic

A forward scan of the reference string determines the next-use distance for every loaded page; the page farthest from future use is evicted. Since it requires complete foreknowledge of future references, it cannot be implemented in a real OS — it is used solely as a performance benchmark.

> **■ Note**
> No realizable algorithm can produce fewer page faults than Optimal for the same input. It serves as the gold-standard lower bound.

Step-by-Step Simulation Trace

Step	Page	Frame 1	Frame 2	Frame 3	Status
1	7	7	-	-	✗ Miss
2	3	7	3	-	✗ Miss
3	0	7	3	0	✗ Miss
4	3	7	3	0	✓ Hit
5	2	7	2	0	✗ Miss
6	1	1	2	0	✗ Miss
7	2	1	2	0	✓ Hit
8	0	1	2	0	✓ Hit
9	7	1	7	0	✗ Miss
10	0	1	7	0	✓ Hit
11	1	1	7	0	✓ Hit
12	2	2	7	0	✗ Miss

■ **Total Page Faults: 7**

MRU — Most Recently Used

How It Works

MRU is the inverse of LRU: it evicts the most recently accessed page on a fault. While counter-intuitive, it can be highly effective for large cyclic or sequential scans where the most recently touched page is the least likely to be revisited soon.

Implementation Logic

A single pointer records the last-accessed page. On a fault, that page is evicted and replaced by the new page. MRU is low-overhead but its performance is highly workload-dependent — it excels on sequential scans and struggles on locality-heavy access.

> ■ **Tip**
> MRU can match or beat LRU on cyclic sequential workloads. For typical locality-heavy patterns, LRU is usually superior.

Step-by-Step Simulation Trace

Step	Page	Frame 1	Frame 2	Frame 3	Status
1	7	7	-	-	✗ Miss
2	3	7	3	-	✗ Miss
3	0	7	3	0	✗ Miss
4	3	7	3	0	✓ Hit
5	2	7	2	0	✗ Miss
6	1	7	1	0	✗ Miss
7	2	7	2	0	✗ Miss
8	0	7	2	0	✓ Hit
9	7	7	2	0	✓ Hit
10	0	0	2	0	✓ Hit
11	1	0	2	1	✗ Miss
12	2	0	2	1	✓ Hit

■ **Total Page Faults: 7**

Second Chance — Clock Algorithm

How It Works

Second Chance is a FIFO enhancement that uses a single reference bit per page. When a page is accessed its bit is set to 1. On replacement the clock hand scans pages in FIFO order: a page with bit=1 is given a second chance (bit cleared, moved to the tail); a page with bit=0 is immediately evicted.

Implementation Logic

A circular queue (the clock) holds all loaded pages. On a fault, the hand advances and examines each page: bit=1 pages are spared (bit zeroed, returned to queue back); the first bit=0 page found is replaced. This gives O(1) overhead with no timestamps — used in Linux's page reclaim subsystem. Bits shown as (1) = referenced.

> ■ **Note**
> Second Chance is deployed in real operating systems (e.g., Linux) as a lightweight LRU approximation with minimal hardware support.

Step-by-Step Simulation Trace

Step	Page	Frame 1	Frame 2	Frame 3	Status
1	7	7(1)	-	-	✗ Miss
2	3	7(1)	3(1)	-	✗ Miss
3	0	7(1)	3(1)	0(1)	✗ Miss
4	3	7(1)	3(1)	0(1)	✓ Hit
5	2	2(1)	3(1)	0(1)	✗ Miss
6	1	2(1)	3(1)	1(1)	✗ Miss
7	2	2(1)	3(1)	1(1)	✓ Hit
8	0	2(1)	0(1)	1(1)	✗ Miss
9	7	2(1)	0(1)	7(1)	✗ Miss
10	0	2(1)	0(1)	7(1)	✓ Hit
11	1	1(1)	0(1)	7(1)	✗ Miss
12	2	1(1)	0(1)	2(1)	✗ Miss

■ **Total Page Faults: 9**

Performance Comparison Summary

d)

Algorithm	Page Faults	Efficiency Rank
Optimal	**7**	**1st — Theoretical minimum (lower boun**
MRU	**7**	**1st — Best practical result for this string**
FIFO	8	3rd
LRU	9	4th
Second Chance	9	4th

Conclusion

■ **Optimal & MRU both achieved 7 page faults** — the minimum for this input. Optimal's count is the mathematical lower bound; MRU matched it because this reference string's cyclic structure penalises recently-used pages, which happen to be poor future candidates.

■ **LRU and Second Chance produced 9 faults** — the highest here. This counter-intuitive result shows that no deterministic policy dominates every workload; performance is always input-dependent.

■ **FIFO sat in the middle at 8 faults.** Notably, Second Chance — designed to improve on FIFO — performed worse on this short, non-repeating string because the reference-bit mechanism deferred evictions that plain FIFO would have made immediately.

- **Production relevance:** Real operating systems (Linux, Windows) predominantly use Clock/Second-Chance variants because their low overhead and strong average-case behaviour across heterogeneous workloads outweigh the worst-case scenarios visible in short single-benchmark tests like this one.